Table des matières

Labo 09 — Effacer le projet, le refaire, le prouver, le livrer 1
Avant de commencer 2
Étape 1 — Une branche pour le capstone 3
Étape 2 — La documentation, complète 3
Étape 3 — La garde 5
Étape 4 — La commande de reconstruction 7
Étape 5 — Mettre de côté, puis effacer 8
Étape 6 — Refaire, depuis une copie neuve du dépôt 8
Étape 7 — Première preuve : les lignes 9
Étape 8 — Deuxième preuve : les témoins 11
Étape 9 — Troisième preuve : l'incident, par les deux bouts 12
Étape 10 — Le `README.md` du dépôt 15
Étape 11 — `LIVRAISON.md`, complet 18
Étape 12 — La CI sur la branche, la fusion, l'étiquette 19
Étape 13 — Le ménage 20
Récapitulatif 20
Dépannage 21

Labo 09 — Effacer le projet, le refaire, le prouver, le livrer

Labo · Module 9/9 — Formation Analytics Engineering

Ce que vous allez faire. Rien de neuf, et c'est le plus exigeant des neuf labos. Vous complétez la documentation du projet, vous écrivez la commande qui le refait de zéro et la garde qui la protège, puis vous effacez tout ce qu'Analytics Engineering a construit dans l'entrepôt depuis le Cours 03. Vous le refaites depuis une copie neuve du dépôt, et vous prouvez qu'il est revenu identique, par ses lignes, par ses témoins et par l'incident de l'usine. Objectif de fin : un dépôt `02_ae` étiqueté `v1.0`, avec un `README.md` qu'une personne qui ne vous connaît pas peut suivre sans vous appeler.

© 2026 iAlexMG. Tous droits réservés.

Deux terminaux servent dans ce labo, plus un navigateur si vous poussez sur GitHub. L'Anaconda Prompt, environnement `formations` actif, sert à dbt, à Git et au petit Python du labo ; il s'ouvre dans `Projet_Boreal/02_ae`, et les étapes qui demandent `Projet_Boreal` ou la copie neuve du dépôt le disent. La ligne de commande DuckDB, ouverte depuis `Projet_Boreal`, sert à effacer, à vérifier et à interroger l'étoile, et elle doit être fermée quand dbt travaille.

La liste de contrôle du cours :

- ☐ La branche `capstone`
- ☐ La documentation complète : 31 modèles, le seed, le snapshot et 14 sources décrits
- ☐ La garde, qui refuse l'entrepôt construit
- ☐ `reconstruire.cmd`, arrêté par la garde, puis commité
- ☐ L'entrepôt mis de côté, puis les quatre schémas du projet effacés
- ☐ Le projet refait depuis une copie neuve du dépôt : `Reconstruction : au vert`
- ☐ Les lignes : 30 relations, 0 écart
- ☐ Les témoins : `PASS=5`, et 465 924 613,14 $ des deux côtés
- ☐ L'incident retrouvé depuis l'après-vente et depuis l'usine
- ☐ Le `README.md` du dépôt
- ☐ `LIVRAISON.md` complet
- ☐ La CI au vert sur la branche, la fusion et l'étiquette `v1.0`
- ☐ Le ménage

Le labo se fait en deux séances. La première va jusqu'à l'étape 6, le projet refait ; la seconde porte les trois preuves et la livraison.

Avant de commencer

Ce labo part du projet tel que le Cours 08 l'a laissé, commité, sur un entrepôt dont la `RAW` va jusqu'au 31 mars 2026. Vérifiez-le dans l'Anaconda Prompt, depuis `Projet_Boreal/02_ae` :

```
dbt build --target prod --exclude snp_concessionnaire
git status --short
```


© 2026 iAlexMG. Tous droits réservés.

`dbt build` doit rendre `PASS=128 WARN=0 ERROR=0` sur 129 nœuds, et `git status` ne doit rien afficher. Un fichier modifié et non commité ferait mentir tout le labo : la reconstruction part d'une copie du dépôt, qui ne voit que ce que Git suit, et elle ne rendrait pas ce que votre entrepôt porte.

Vérifiez aussi qu'aucun schéma de travail ne traîne, dans la ligne de commande DuckDB, depuis `Projet_Boreal` :

```
select schema_name from duckdb_schemas() where database_name = 'boreal' order by 1;
```

Attendu : `argent`, `main`, `marts`, `pipeline`, `presentation`, `raw`, `snapshots` et `staging`. S'il reste un `dev_*`, fermez DuckDB et lancez `dbt run-operation nettoyer_dev` (Cours 08).

Rien ne s'installe dans ce cours. Un compte GitHub reste facultatif, comme au Cours 08 : sans lui, l'étape 12 répète la CI sur votre poste.

Étape 1 — Une branche pour le capstone

Théorie : `Theorie_08_AE.md`, section 1.

Tout ce que ce labo ajoute au dépôt passe par une branche, et la CI la jugera à la fin, comme n'importe quel changement proposé :

```
git switch -c capstone
```

Le travail d'un capstone ressemble à celui d'une vraie livraison : plusieurs fichiers, plusieurs commits, et une demande de tirage qui les présente ensemble. La branche principale reste à l'état du Cours 08 jusqu'à la fusion.

Étape 2 — La documentation, complète

Théorie : `Theorie_09_AE.md`, section 7.

Le Cours 08 l'a mesuré : le manifeste ne dit que ce que le projet y écrit. Comptez ce qui manque encore :

```
dbt parse
python -c "import json; m = json.load(open('target/manifest.json', encoding='utf-8')); print([n['name'] for n in
m['nodes'].values() if n['resource_type'] in ('model', 'seed', 'snapshot') and not n['description']], [s['name'] for s in
m['sources'].values() if not s['description']])"
```

Deux listes sortent. La première nomme dix-huit objets : seize modèles, les trois `int_*` et les treize vues de `staging`, dont le modèle de contrôle, plus le snapshot et le seed, que le Cours 08 ne comptait pas. La seconde est vide : chaque source a reçu sa description le jour où elle a été déclarée, aux Cours 03 à 08. Un agent qui lirait ce manifeste, ou un analyste qui ouvrirait la documentation, ne saurait de ces dix-huit objets que leur nom.

Une description dit le grain et ce que le modèle a déjà fait à la source, pas son SQL. Créez `models/staging/_staging.yml` :

© 2026 iAlexMG. Tous droits réservés.

```yaml
version: 2

models:
  - name: stg_actifs
    description: "Un actif de l'usine par ligne, zone ou équipement, tel que le registre le livre."
  - name: stg_articles
    description: "Un article par ligne, dans son état courant, sa gamme et son type décodés."
  - name: stg_clients
    description: >
      Un client par ligne, dans sa version la plus récente. Aucune donnée personnelle directe :
      la ville, la province, la langue et le drapeau d'effacement.
  - name: stg_commandes
    description: >
      Un en-tête de commande par ligne, dans son état courant : le lot de mars 2023 livré deux
      fois et les commandes modifiées n'y comptent qu'une fois.
  - name: stg_concessionnaires
    description: "Un concessionnaire par ligne, son palier décodé, dans l'état du dernier extrait."
  - name: stg_enregistrements
    description: "Un véhicule vendu au détail et enregistré par ligne, tel que le portail le livre."
  - name: stg_etiquettes
    description: "Un point mesuré de l'historien par ligne, avec l'actif et la grandeur qu'il mesure."
  - name: stg_postes
    description: >
      Un poste de commande par ligne, dans son état courant. Un poste refusé porte son motif ;
      la vue ne l'écarte pas, elle le rend reconnaissable.
  - name: stg_reclamations
    description: >
      Une réclamation par ligne, dans son dernier statut. Son historique reste en JSON, et
      int_jalons_reclamations le déplie.
  - name: stg_regions
    description: "Le libellé français de chaque province, tronqué à vingt caractères comme dans l'ERP."
  - name: stg_sorties
    description: >
      Une sortie d'un véhicule connecté par ligne, dédoublonnée et aplatie. Seule la liste des
      codes levés reste en JSON, et fct_codes_sortie la déplie.
  - name: stg_vehicules
    description: "Un véhicule assemblé par ligne, identifié par son numéro de série, tel que l'ERP le connaît."
  - name: ctl_commandes_sans_concessionnaire
    description: >
      Le premier contrôle du projet : les commandes dont le concessionnaire manque au
      référentiel. Attendu : aucune ligne.
```


© 2026 iAlexMG. Tous droits réservés.

Puis `models/intermediate/_intermediate.yml` :

```yaml
version: 2

models:
  - name: int_jalons_reclamations
    description: "L'historique de chaque réclamation, déplié en une ligne par changement de statut. Lu par fct_reclamations."
  - name: int_lignes_enrichies
    description: "Les postes actifs, enrichis du coût et de la marge, pour que fct_ventes reste mince."
  - name: int_usine_par_actif
    description: >
      Les séries horaires de l'usine rangées par actif : une ligne par actif et par heure, avec
      la part en marche, la puissance et la vibration. Lu par fct_disponibilite_actifs.
```

Ces trois modèles sont `ephemeral` : rien dans l'entrepôt ne les montre, et seule la documentation dit qu'ils existent. C'est une raison de plus de les décrire, pas de moins.

Le snapshot et le seed se décrivent dans leur propre dossier, que dbt lit comme `models/`. Créez `snapshots/_snapshots.yml` :

```yaml
version: 2

snapshots:
  - name: snp_concessionnaire
    description: >
      L'histoire des paliers des concessionnaires : une version par changement de palier, datée de
      la fin du mois de l'extrait. Elle ne se refait que si la RAW garde ses extraits mensuels.
```

Et `seeds/_seeds.yml` :

```yaml
version: 2

seeds:
  - name: anomalies
    description: "Les cinq codes d'anomalie, avec leur libellé et leur système, tirés du guide GUI-CODES du corpus."
```

Le snapshot est la seule table du projet qui porte une histoire, et personne ne l'avait décrite. La section 3 de la théorie dit pourquoi sa description compte plus que les autres.

Relancez les deux commandes du début. Attendu : `[] []`, deux listes vides. Écrivez aussi, en tête de chaque fichier neuf, un bloc de commentaires qui dit à quoi il sert, comme le font tous les fichiers du `Solution/`.

Étape 3 — La garde

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_09_AE.md`, section 4.

La reconstruction que vous allez écrire rejoue les quatre lots de l'histoire des paliers. Lancée sur un entrepôt qui porte déjà cette histoire, elle l'abîmerait sans bruit : mesuré, le snapshot passe de 205 versions à 303, dont 49 qui finissent avant de commencer. Avant d'écrire la commande, on écrit ce qui l'empêche de tourner au mauvais endroit. Créez `macros/exiger_entrepot_neuf.sql` :

```
{% macro exiger_entrepot_neuf() %}
    {% set schemas = [] %}
    {% for dossier in ['staging', 'marts', 'presentation'] %}
        {% do schemas.append(generate_schema_name(dossier, none) | trim) %}
    {% endfor %}
    {% do schemas.append('snapshots') %}

    {% set requete %}
        select table_schema, count(*) as objets
        from information_schema.tables
        where table_schema in ('{{ schemas | join("', '") }}')
        group by table_schema
        order by table_schema
    {% endset %}
    {% set deja_la = run_query(requete) %}

    {% for ligne in deja_la.rows %}
        {{ log('Deja construit : ' ~ ligne[0] ~ ', ' ~ ligne[1] ~ ' objets', info=True) }}
    {% endfor %}
    {% if deja_la.rows | length > 0 %}
        {{ exceptions.raise_compiler_error(
            "exiger_entrepot_neuf : l'entrepot porte deja le projet. Une reconstruction part d'un "
            ~ "entrepot qui n'a que la RAW et l'argent."
        ) }}
    {% endif %}
    {{ log('Entrepot neuf : rien dans ' ~ schemas | join(', '), info=True) }}
{% endmacro %}
```

Rien ici n'est neuf. Les noms des schémas viennent de `generate_schema_name`, comme dans `nettoyer_dev` au Cours 08, et la garde suit donc la convention le jour où elle change. Le schéma des snapshots, lui, s'ajoute en dur : sa configuration le fixe par `target_schema`, qui ne passe pas par la macro. `run_query` exécute la requête et rend un tableau, dont `rows` porte les lignes.

Essayez-la sur votre entrepôt, qui porte le projet entier :

```
dbt run-operation exiger_entrepot_neuf --target prod
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : quatre lignes, `Deja construit : marts, 15 objets`, `presentation, 1 objets`, `snapshots, 1 objets` et `staging, 13 objets`, puis l'erreur de la macro et le code 1. Rien n'a été touché : la garde ne fait que regarder.

Étape 4 — La commande de reconstruction

Théorie : `Theorie_09_AE.md`, section 4.

Refaire le projet demande neuf commandes, dans un ordre qui ne se discute pas. Les taper à la main un jour de panne, c'est en oublier une ou en inverser deux. Elles s'écrivent donc une fois, dans `reconstruire.cmd`, à la racine du dépôt :

```
@echo off
rem reconstruire.cmd : le projet refait de zero, depuis la racine du depot (Cours 09, etape 4).
rem
rem L'entrepot ne doit porter que ce que Data Engineering livre, la RAW et l'argent : la garde le
rem verifie avant tout le reste. Les lots du snapshot se rejouent dans l'ordre, du plus ancien au
rem plus recent, puis l'etat courant ; la strategie check compare des valeurs, pas des dates.
rem Chaque commande arrete le script si elle echoue, et le code de sortie le dit.
dbt deps || exit /b 1
dbt run-operation exiger_entrepot_neuf --target prod || exit /b 1
dbt snapshot --target prod --vars "{lot: 2022-01}" || exit /b 1
dbt snapshot --target prod --vars "{lot: 2024-06}" || exit /b 1
dbt snapshot --target prod --vars "{lot: 2024-07}" || exit /b 1
dbt snapshot --target prod --vars "{lot: 2025-12}" || exit /b 1
dbt snapshot --target prod || exit /b 1
dbt build --target prod --exclude snp_concessionnaire || exit /b 1
dbt docs generate --target prod || exit /b 1
echo Reconstruction : au vert
```

Trois choses à lire dans ce fichier.

La cinquième passe du snapshot n'a pas de lot : elle capture l'état courant, le dernier extrait livré, comme le ferait un ordonnanceur chaque nuit. Le Cours 05 s'arrêtait au lot de décembre 2025 ; celle-ci amène l'histoire jusqu'au dernier mois chargé, quel qu'il soit.

`|| exit /b 1` suit chaque commande, comme dans le script de la CI que la question P5 du Cours 08 faisait écrire. Sans lui, le script continuerait après un échec, et la dernière ligne annoncerait un vert que rien ne justifie.

Et `dbt docs generate` ferme la marche : la documentation fait partie de ce que le projet livre, et elle se refait avec lui.

Lancez-le sur votre entrepôt, qui porte le projet :

```
reconstruire.cmd
```


© 2026 iAlexMG. Tous droits réservés.

Il s'arrête à la deuxième commande, sur la garde, avec les quatre lignes de l'étape 3 et le code 1. Aucun snapshot n'a tourné. C'est la preuve que la garde est branchée, et c'est la dernière fois que vous lancez ce script ailleurs que sur un entrepôt neuf.

Commitez ce que les étapes 2 à 4 ont écrit. La copie neuve de l'étape 6 ne verra que cela :

```
git add models snapshots seeds macros reconstruire.cmd
git commit -m "Capstone : la documentation, la garde et la reconstruction"
```

Étape 5 — Mettre de côté, puis effacer

Théorie : `Theorie_09_AE.md`, sections 1 et 3.

La première preuve demandera l'entrepôt d'avant, et il n'existera plus une fois effacé. Fermez la ligne de commande DuckDB, puis, dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
copy plateforme\entrepot\boreal.duckdb plateforme\entrepot\boreal_avant.duckdb
```

Une copie de fichier, pas une exportation : environ 90 Mo, en une seconde. Le nom compte, et la section 5 de la théorie dit pourquoi.

Effacez maintenant ce qu'Analytics Engineering a construit, et rien d'autre. Ouvrez la ligne de commande DuckDB, depuis `Projet_Boreal` :

```
duckdb plateforme/entrepot/boreal.duckdb
```

```
drop schema staging cascade;
drop schema marts cascade;
drop schema presentation cascade;
drop schema snapshots cascade;
select schema_name from duckdb_schemas() where database_name = 'boreal' order by 1;
```

Attendu : `argent`, `main`, `pipeline` et `raw`. Il reste ce que Data Engineering a livré et que vous n'avez jamais écrit : la `RAW`, les séries de l'usine et le journal des chargements. Le schéma `main` est celui que DuckDB crée pour tout fichier, et il est vide. Fermez DuckDB par `.exit`.

Piège — le snapshot n'est pas une table comme les autres. Les quatorze tables de `marts` se refont par `dbt build`, à partir de la `RAW`. L'histoire des paliers ne se refait que parce que la `RAW` garde chaque extrait mensuel du référentiel, 52 lots à l'état de mars 2026 : effacer `snapshots` sur un projet dont la source ne garde que l'état courant, c'est perdre cette histoire pour de bon. La section 3 de la théorie le mesure.

Étape 6 — Refaire, depuis une copie neuve du dépôt

Théorie : `Theorie_09_AE.md`, sections 1 et 4.

La reconstruction ne part pas de votre dossier de travail, qui porte des paquets installés, une documentation, des journaux et peut-être un fichier oublié. Elle part de ce que Git suit, comme le ferait un collègue. Depuis `Projet_Boreal` :

© 2026 iAlexMG. Tous droits réservés.

```
git clone 02_ae 02_ae_neuf
cd 02_ae_neuf
git branch --show-current
```

La copie est sur `capstone`, la branche où vous travaillez. Elle n'a ni `dbt_packages/`, ni `target/`, ni `logs/`. Votre profil personnel, lui, est hors du dépôt et sert tel quel : c'est le même entrepôt.

```
reconstruire.cmd
```

Attendu, en un peu plus d'une minute : `dbt_utils` installé en 1.4.1 ; la garde qui répond `Entrepot neuf : rien dans staging, marts, presentation, snapshots` ; cinq passes du snapshot, chacune `PASS=1` ; la construction, `PASS=128 WARN=0 ERROR=0` sur 129 nœuds ; la documentation, `Catalog written to ...\02_ae_neuf\target\catalog.json` ; et la dernière ligne, `Reconstruction : au vert`. Chaque passe du snapshot répète l'avertissement du Cours 05 sur le type `DATE` de `updated_at`, sans conséquence.

Le temps se répartit mal, et c'est instructif : 68 secondes mesurées, dont une dizaine pour la construction de 129 nœuds et quarante pour les cinq passes du snapshot. Chaque passe dure huit secondes, dont moins d'une pour traiter 156 lignes ; le reste est le démarrage de dbt, qui relit le projet à chaque commande.

Vérifiez l'histoire, dans DuckDB, depuis `Projet_Boreal`, dbt ayant fini :

```
select count(*) as versions,
       count(distinct code_concessionnaire) as concessionnaires,
       count(*) filter (where dbt_valid_to is not null) as fermees,
       count(*) filter (where dbt_valid_to < dbt_valid_from) as fin_avant_debut
from snapshots.snp_concessionnaire;
```

Attendu : 205 versions, 156 concessionnaires, 49 fermées, 0 qui finit avant de commencer. La cinquième passe n'a rien ajouté : aucun palier ne change dans les trois lots de 2026.

C'est la fin de la première séance. Le projet est refait ; il reste à prouver qu'il est le même.

Étape 7 — Première preuve : les lignes

Théorie : `Theorie_09_AE.md`, section 5.

Revenez dans votre dépôt de travail, `Projet_Boreal/02_ae` : les preuves n'y sont pas encore, et elles s'écrivent là. Créez `outils/comparer_reconstruction.py` :

```
import sys

import duckdb

SCHEMAS = ("staging", "marts", "presentation", "snapshots")
```


© 2026 iAlexMG. Tous droits réservés.

```
avant, apres = sys.argv[1], sys.argv[2]
c = duckdb.connect(apres, read_only=True)
c.execute(f"attach '{avant}' as avant (read_only)")
refait = c.sql("select current_database()").fetchone()[0]


def relations(base):
    lignes = c.execute(
        "select table_schema, table_name, table_type from information_schema.tables "
        "where table_catalog = ? and table_schema in ? order by 1, 2",
        [base, list(SCHEMAS)],
    ).fetchall()
    return {(s, t): ty for s, t, ty in lignes}


def definition(base, schema, vue):
    return c.execute(
        "select sql from duckdb_views() where database_name = ? and schema_name = ? "
        "and view_name = ?",
        [base, schema, vue],
    ).fetchone()[0]


r_avant, r_apres = relations("avant"), relations(refait)
ecarts = 0
print(f"{'relation':42} {'avant':>8} {'apres':>8} {'manquent':>9} {'en trop':>8}")
for s, t in sorted(set(r_avant) | set(r_apres)):
    nom = f"{s}.{t}"
    if (s, t) not in r_avant or (s, t) not in r_apres:
        ecarts += 1
        print(f"{nom:42} presente d'un seul cote")
    elif r_apres[(s, t)] == "VIEW":
        identique = definition("avant", s, t) == definition(refait, s, t)
        ecarts += 0 if identique else 1
        print(f"{nom:42} vue, definition {'identique' if identique else 'DIFFERENTE'}")
    else:
        a = c.sql(f'select count(*) from avant."{s}"."{t}"').fetchone()[0]
        b = c.sql(f'select count(*) from "{refait}"."{s}"."{t}"').fetchone()[0]
        manquent = c.sql(
            f'select count(*) from (select * from avant."{s}"."{t}" '
            f'except all select * from "{refait}"."{s}"."{t}")'
        ).fetchone()[0]
```


© 2026 iAlexMG. Tous droits réservés.

```
            en_trop = c.sql(
                f'select count(*) from (select * from "{refait}"."{s}"."{t}" '
                f'except all select * from avant."{s}"."{t}")'
            ).fetchone()[0]
            ecarts += manquent + en_trop
            print(f"{nom:42} {a:>8} {b:>8} {manquent:>9} {en_trop:>8}")

    print(f"{len(r_apres)} relations refaites, {len(r_avant)} avant : {ecarts} ecart(s)")
    sys.exit(1 if ecarts else 0)
```

Le script ouvre l'entrepôt refait en lecture seule et y attache celui d'avant sous le nom `avant`. Il prend les relations des quatre schémas des deux côtés, et une relation présente d'un seul côté est un écart : une table qui manque au refait, ou une table que rien dans le dépôt ne fabrique plus. Une table se compare par ses lignes, dans les deux sens, par `except all`, qui compte les exemplaires : une ligne en double d'un seul côté se voit.

Une vue se compare par sa définition, et la section 5 de la théorie dit pourquoi ses lignes ne prouveraient rien. Depuis `Projet_Boreal`, la ligne de commande DuckDB fermée :

```
python 02_ae/outils/comparer_reconstruction.py plateforme/entrepot/boreal_avant.duckdb plateforme/entrepot/boreal.duckdb
```

Attendu, en une ou deux secondes, trente lignes dont voici le début et la fin :

```
relation                                    avant    apres  manquent  en trop
marts.anomalies                                 5        5         0        0
marts.dim_actif                                23       23         0        0
marts.dim_anomalie                              5        5         0        0
marts.dim_client                            24719    24719         0        0
marts.dim_concessionnaire                     205      205         0        0
marts.dim_date                               1551     1551         0        0
...
marts.fct_ventes                            80618    80618         0        0
presentation.rpt_ventes_par_palier        vue, definition identique
snapshots.snp_concessionnaire                 205      205         0        0
staging.ctl_commandes_sans_concessionnaire vue, definition identique
...
30 relations refaites, 30 avant : 0 ecart(s)
```

Seize tables égales ligne pour ligne, quatorze vues à la définition identique, et le code 0. Même les moyennes horaires de l'usine, des nombres à virgule flottante, reviennent au dernier chiffre près : la même requête, sur les mêmes données, avec la même version de DuckDB.

Étape 8 — Deuxième preuve : les témoins

Théorie : `Theorie_09_AE.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

La première preuve compare le refait à l'avant, et ne peut donc rien dire de ce qui était déjà faux avant. Les témoins comparent le projet à autre chose que lui-même. Depuis `Projet_Boreal/02_ae` :

```
dbt test --target prod --select test_type:singular
```

Attendu : `PASS=5`. Ce sont les cinq tests singuliers du projet, écrits aux Cours 05 à 07, et chacun oppose l'étoile à un chemin qui ne passe pas par elle.

Le test	Ce qu'il compare
`assert_reconciliation_ventes`	la valeur nette du fait, commande par commande, à celle des en-têtes de l'ERP
`assert_production_contre_compteur`	les véhicules de `fct_production`, jour par jour, au compteur des convoyeurs de l'usine
`assert_refus_hors_garantie`	les refus pour hors garantie à la règle recalculée depuis les dates de garantie
`assert_jalons_dans_l_ordre`	l'ordre des jalons de chaque réclamation
`assert_compteur_heures_moteur`	la hausse du compteur d'heures moteur à la durée des sorties

Et le premier, en clair, dans DuckDB, depuis `Projet_Boreal` :

```
select
    (select sum(valeur_nette) from marts.fct_ventes) as fait,
    (select sum(valeur_nette) from staging.stg_commandes) as erp_entetes;
```

Attendu : 465 924 613,14 des deux côtés, le chiffre du Cours 07. Il a traversé une reconstruction de zéro sans perdre un cent.

Étape 9 — Troisième preuve : l'incident, par les deux bouts

Théorie : `Theorie_09_AE.md`, sections 5 et 6.

La dernière preuve ne compare rien à rien : elle vérifie que l'étoile refaite répond encore à la question pour laquelle elle existe. Chez Boréal, c'est l'incident du compresseur d'air, que les Cours 06 et 07 ont suivi. On le cherche cette fois sans en connaître la date, une fois depuis l'après-vente, une fois depuis l'usine, et les deux chemins doivent tomber au même endroit.

Depuis l'après-vente d'abord. Un responsable de la garantie ne connaît que ses réclamations ; il les range par semaine d'assemblage des motoneiges vendues :

```
select date_trunc('week', v.date_assemblage)::date as semaine,
       count(*) as vendues,
       count(*) filter (where v.cle_vehicule in (
           select r.cle_vehicule from marts.fct_reclamations as r where r.cause = 'REFROIDISSEMENT'
```


© 2026 iAlexMG. Tous droits réservés.

```
       )) as refroidissement
from marts.fct_enregistrements as e
inner join marts.dim_vehicule as v on v.cle_vehicule = e.cle_vehicule
where v.gamme = 'Motoneige'
  and v.date_assemblage between date '2025-08-01' and date '2025-12-31'
group by 1 order by 1;
```

Vingt et une semaines. Dix-neuf d'entre elles portent de zéro à cinq réclamations de refroidissement, 24 en tout ; les deux autres ne ressemblent à rien d'autre :

Semaine d'assemblage	Vendues	Refroidissement
2025-10-06	120	2
2025-10-13	95	1
2025-10-20	119	16
2025-10-27	118	18
2025-11-03	119	0

Depuis l'usine ensuite. Un responsable de la maintenance ne connaît que ses machines ; il cherche les jours où la vibration du compresseur 2, en marche, sort de sa bande habituelle, qu'il fixe au 95e centile de tout l'instantané. `quantile_cont` calcule ce centile par interpolation (Cours 09 de SQL, section 8) :

```
with jours as (
    select f.cle_date, avg(f.vibration_moyenne) filter (where f.part_en_marche > 0.5) as v
    from marts.fct_disponibilite_actifs as f
    inner join marts.dim_actif as a on a.cle_actif = f.cle_actif
    where a.actif_id = 'UTL-CMP2'
    group by 1
)
select min(cle_date), max(cle_date), count(*)
from jours
where v > (select quantile_cont(v, 0.95) from jours);
```

Attendu : du 2025-10-21 au 2025-10-28, six jours ouvrés. Sur les 120 jours où le compresseur a tourné, sa vibration médiane vaut 2,47 millimètres par seconde, et le 95e centile 3,28.

Les deux bouts se rejoignent par la date, la dimension conforme que les deux étoiles partagent. Jour par jour, la vibration du compresseur et les réclamations des motoneiges assemblées ce jour-là :

```
with usine as (
    select
        f.cle_date as jour,
```


© 2026 iAlexMG. Tous droits réservés.

```
        round(avg(f.vibration_moyenne) filter (where f.part_en_marche > 0.5), 2) as vibration
    from marts.fct_disponibilite_actifs as f
    inner join marts.dim_actif as a on a.cle_actif = f.cle_actif
    where a.actif_id = 'UTL-CMP2'
    group by 1
),

ligne as (
    select
        p.cle_date_assemblage as jour,
        count(*) as assemblees,
        count(*) filter (where p.cle_vehicule in (
            select r.cle_vehicule from marts.fct_reclamations as r
            where r.cause = 'REFROIDISSEMENT'
        )) as refroidissement
    from marts.fct_production as p
    inner join marts.dim_actif as a on a.cle_actif = p.cle_actif
    where a.actif_id = 'LMN'
    group by 1
)

select u.jour, u.vibration, l.assemblees, l.refroidissement
from usine as u
inner join ligne as l on l.jour = u.jour
where u.jour between date '2025-10-14' and date '2025-10-31'
order by u.jour;
```

Jour	Vibration	Assemblées	Refroidissement
2025-10-14	2,57	24	0
2025-10-15	2,45	24	1
2025-10-16	2,48	24	0
2025-10-17	2,70	24	0
2025-10-20	3,26	24	2
2025-10-21	3,75	24	1
2025-10-22	4,48	24	3

© 2026 iAlexMG. Tous droits réservés.

Jour	Vibration	Assemblées	Refroidissement
2025-10-23	4,74	24	5
2025-10-24	5,70	24	5
2025-10-27	7,68	24	10
2025-10-28	8,47	24	7
2025-10-29	vide	24	1
2025-10-30	2,42	24	0
2025-10-31	2,48	24	0

Les deux colonnes montent ensemble, et retombent ensemble le lendemain du déclenchement : le 29, le compresseur 2 ne tourne pas, et c'est l'appoint qui porte l'usine. Personne n'a écrit cette table : elle sort de trois faits bâtis à trois cours différents, qui partagent une date et un actif. C'est la matrice de bus du Cours 02, qui tient sur un entrepôt refait de zéro.

Étape 10 — Le `README.md` du dépôt

Théorie : `Theorie_09_AE.md`, sections 2 et 7.

Votre dépôt a neuf cours d'histoire, et aucun fichier ne dit à quelqu'un d'autre par où commencer. Depuis `Projet_Boreal/02_ae` :

```
notepad README.md
```

Le début : ce qu'est le dépôt, ce qu'il livre, ce qui n'y est pas, ce qu'il suppose.

```
# 02_ae — l'entrepôt analytique de Véhicules Boréal inc.

Le projet dbt d'Analytics Engineering de Boréal, une entreprise fictive qui fabrique des
motoneiges, des motomarines et des véhicules hors route à l'usine de Val-des-Aulnes. Il reçoit la
couche `RAW` que Data Engineering charge, la rend lisible, en tire quatre étoiles qui partagent
leurs dimensions, et prouve qu'elles disent vrai. Écrit au fil des neuf cours de la formation
Analytics Engineering du parcours Data_Domain, sur dbt et DuckDB.

## Ce qu'il livre

| Produit | Où | Pour qui |
|---|---|---|
| L'étoile des ventes : `fct_ventes`, au grain de la ligne de commande, sous contrat | schéma `marts` de
```


© 2026 iAlexMG. Tous droits réservés.

`plateforme/entrepot/boreal.duckdb` | Business Intelligence, Data Science |
L'étoile de l'après-vente : `fct_enregistrements`, `fct_reclamations`	schéma `marts`	Business Intelligence, Data Science, AI Engineering
L'étoile de l'usine : `fct_disponibilite_actifs`, `fct_production`	schéma `marts`	Business Intelligence, Data Science, Machine Learning Engineering
La télémétrie : `fct_sorties`, `fct_codes_sortie`, en incrémental	schéma `marts`	Data Science, Machine Learning Engineering
Sept dimensions conformes, dont `dim_concessionnaire` en type 2	schéma `marts`	toutes
L'histoire des paliers	schéma `snapshots`	`dim_concessionnaire`
Le résumé des ventes par palier	schéma `presentation`	le tableau de bord que `_exposures.yml` déclare
La documentation et le `manifest.json`	`target/`, et l'artefact de la CI	tous, et les agents d'AI Engineering

Le détail de chaque produit, son grain, ses comptes et ce qu'il faut savoir avant de s'en servir, est dans `LIVRAISON.md`. Sur la génération jusqu'à 2026-03 : 31 modèles, 30 objets bâtis, dont 16 tables et 445 795 lignes ; 98 tests de données et un test unitaire ; 465 924 613,14 $ au fait des ventes, au cent près de l'ERP.

Ce qui n'est pas ici

- Les données. La `RAW` et l'argent vivent dans l'entrepôt, à côté du dépôt, dans `Projet_Boreal/plateforme/` : Data Engineering les charge, ou le kit 02 les monte. Tout ce que ce projet en tire se refait.
- Le profil de connexion, `profiles.yml`, dans le dossier `.dbt` de votre répertoire personnel. Seul celui de la CI, `ci/profiles.yml`, entre au dépôt : il ne porte qu'un nom de fichier.
- Ce que dbt fabrique, `target/`, `dbt_packages/` et `logs/`, que le `.gitignore` écarte.

Avant de commencer

- `Projet_Boreal/`, avec un entrepôt dont les schémas `raw` et `argent` sont chargés jusqu'à 2026-03. Sans le projet de Data Engineering, depuis le dossier `Data_Domain` : `python _univers/kits/demarrer.py --formation 02 --projet Projet_Boreal --jusqua 2026-03`.
- L'environnement conda `formations` : Python 3.11, dbt-core 1.12.3, dbt-duckdb 1.11.0, sqlfluff 4.3.0, et la ligne de commande DuckDB.
- Un `profiles.yml` qui déclare le profil `boreal_ae` et ses deux cibles, `dev` et `prod`, chacune avec le chemin absolu de l'entrepôt.

La suite : refaire, prouver, vérifier.

Refaire le projet

Sur un entrepôt qui n'a que la `RAW` et l'argent, depuis la racine du dépôt, dans l'Anaconda Prompt :

© 2026 iAlexMG. Tous droits réservés.

```bash
reconstruire.cmd
```

Neuf commandes, en un peu plus d'une minute : les paquets, une garde qui refuse un entrepôt déjà construit, les quatre lots de l'histoire des paliers dans l'ordre puis l'état courant, la construction et ses tests, la documentation. Attendu : `PASS=128` à la construction, puis `Reconstruction : au vert`. Le script s'arrête à la première commande qui échoue.

Prouver

Trois preuves, depuis `Projet_Boreal`, la ligne de commande DuckDB fermée. Si l'entrepôt d'avant a été mis de côté, la comparaison ligne à ligne, dans les deux sens :

```bash
python 02_ae/outils/comparer_reconstruction.py plateforme/entrepot/boreal_avant.duckdb plateforme/entrepot/boreal.duckdb
```

Attendu : 30 relations, 0 écart. Puis, depuis `02_ae`, les cinq témoins indépendants du projet :

```bash
dbt test --target prod --select test_type:singular
```

La valeur des ventes contre l'ERP, la production contre le compteur du convoyeur, les refus hors garantie contre la règle, les jalons des réclamations dans l'ordre, le compteur d'heures moteur des véhicules connectés : `PASS=5`. La troisième preuve est l'incident du compresseur d'air d'octobre 2025, qui se retrouve depuis l'usine et depuis l'après-vente ; les requêtes sont dans `LIVRAISON.md`, section 12.

La vérification continue

`.github/workflows/ci.yml` tourne à chaque demande de tirage vers `main` et à chaque poussée sur `main` : le linteur, puis `dbt build` sur un entrepôt jetable qui n'a que la structure des sources. Sur votre poste, depuis une copie neuve du dépôt :

```bash
dbt deps
sqlfluff lint models tests snapshots
python -c "import duckdb; duckdb.connect('ci.duckdb').execute(open('ci/structure_sources.sql', encoding='utf-8').read())"
dbt build --target ci --profiles-dir ci
```


© 2026 iAlexMG. Tous droits réservés.

```
Attendu : `All Finished!` au linteur, puis `PASS=129` sur 130 nœuds.
```

Et la carte du dépôt :

```
## Le dépôt

| Dossier ou fichier | Ce qu'il porte |
|---|---|
| `models/staging/` | une vue par table reçue : renommer, typer, décoder, tirer l'état courant ; les sources et leur fraîcheur |
| `models/intermediate/` | trois modèles `ephemeral`, qui gardent les faits minces |
| `models/marts/` | les sept dimensions et les sept faits, leurs tests, le contrat de `fct_ventes` et l'`exposure` du tableau de bord |
| `models/presentation/` | le résumé que lit le tableau de bord |
| `snapshots/` | l'histoire des paliers des concessionnaires |
| `seeds/` | les cinq codes d'anomalie, que seul un guide du corpus nomme |
| `tests/` | les cinq témoins indépendants |
| `macros/` | le routage des schémas, le ménage des schémas de travail, la garde de la reconstruction |
| `ci/`, `.github/`, `.sqlfluff` | la vérification continue |
| `outils/` | la lecture du manifeste, la comparaison de deux entrepôts |
| `reconstruire.cmd` | le projet refait de zéro |
| `LIVRAISON.md` | ce que le projet livre, produit par produit |
```

Relisez-le comme le lirait quelqu'un qui ne vous connaît pas. Il ne raconte pas le code, que les en-têtes des fichiers expliquent : il dit quoi avoir avant de commencer, quoi taper, dans quel ordre, et ce qui doit sortir. Il dit aussi ce qui n'est pas dans le dépôt, et pourquoi. Le détail de chaque produit reste dans `LIVRAISON.md`, que la formation tient à jour depuis le Cours 04.

Étape 11 — `LIVRAISON.md`, complet

Théorie : `Theorie_09_AE.md`, section 7.

Trois retouches, et le document est fini.

La section 1 donne la commande de reconstruction : remplacez la liste des sept commandes par `reconstruire.cmd`, en gardant la liste des neuf commandes qu'il enchaîne, et dites ce que fait la cinquième passe du snapshot.

Ajoutez deux sections avant la dernière. La première dit la reconstruction et ses preuves : l'entrepôt qu'elle suppose, la garde et ce qu'elle refuse, la durée, l'état livré table par table, les trois preuves et leurs attendus, et ce qu'une reconstruction ne refait pas, l'histoire des paliers sans les extraits gardés. La seconde donne les requêtes de l'incident, depuis l'après-vente et depuis l'usine, avec leurs résultats : c'est la section 12 que le `README.md` annonce.

© 2026 iAlexMG. Tous droits réservés.

Reprenez enfin la section finale, qui dit ce que la livraison ne porte pas encore. Deux lignes tombent : le snapshot qui s'arrêtait en décembre 2025, puisque la cinquième passe l'amène au dernier lot, et la documentation incomplète. Les autres restent, et la section dit maintenant ce qu'un consommateur doit savoir avant de s'appuyer sur la version 1.0.

Étape 12 — La CI sur la branche, la fusion, l'étiquette

Théorie : `Theorie_09_AE.md`, section 7.

Commitez les preuves et la livraison :

```
git add outils/comparer_reconstruction.py README.md LIVRAISON.md
git commit -m "Capstone : les preuves, le README et la livraison"
git log --oneline main..capstone
```

Deux commits sur la branche, ceux des étapes 4 et 12.

Si vous avez un dépôt sur GitHub, proposez la branche comme au Cours 08, puis suivez la CI :

```
git push -u origin capstone
gh pr create --fill
gh pr checks --watch
```

Attendu : la tâche `verifier` au vert en moins d'une minute. Elle a passé au linteur les SQL de la branche et construit ses 130 nœuds sur la structure vide des sources ; la garde, la documentation et le `README.md` y sont entrés sans rien casser. Fusionnez, puis ramenez la branche principale :

```
gh pr merge --merge --delete-branch
git switch main
git pull
```

Sans compte GitHub, répétez la CI sur une copie neuve, comme à l'étape 10 du Cours 08, puis fusionnez sur votre poste :

```
git switch main
git merge --no-ff capstone -m "Capstone : le projet refait de zero, prouve et livre"
git branch -d capstone
```

Étiquetez l'état livré, et régénérez la documentation de votre dossier de travail, que la reconstruction avait écrite dans la copie :

```
git tag -a v1.0 -m "Le projet dbt de Boreal, refait de zero et prouve"
git log --oneline --decorate -1
dbt docs generate --target prod
```

Une étiquette annotée, `-a`, porte un auteur, une date et un message, comme un commit. `v1.0` nomme cet état du dépôt, et le nommera encore quand `main` aura avancé : c'est lui que les formations suivantes citeront. Si votre dépôt a un serveur distant, l'étiquette ne part pas avec `git push` : elle demande `git push origin v1.0`.

© 2026 iAlexMG. Tous droits réservés.

Étape 13 — Le ménage

La copie neuve et l'entrepôt d'avant ont servi, et ils pèsent. Depuis `Projet_Boreal`, DuckDB fermé :

```
rmdir /s /q 02_ae_neuf
del plateforme\entrepot\boreal_avant.duckdb
```

Rien de ce que vous effacez ici n'est une donnée : la copie se refait par `git clone`, l'entrepôt d'avant a fini son travail de témoin, et l'entrepôt refait est celui que les formations suivantes liront.

Récapitulatif

L'état du projet à la fin de la formation :

Ce qui est en place	Vérification
Le dépôt	63 fichiers suivis, étiqueté `v1.0` ; `git status --short` ne rend rien
La documentation	31 modèles, le seed, le snapshot et 14 sources décrits ; `index.html`, `manifest.json` et `catalog.json` dans `target/`
La garde	`dbt run-operation exiger_entrepot_neuf --target prod` refuse l'entrepôt construit, code 1
La reconstruction	`reconstruire.cmd` sur un entrepôt neuf : neuf commandes, un peu plus d'une minute, `Reconstruction : au vert`
L'histoire des paliers	205 versions, 156 concessionnaires, 49 fermées, aucune qui finit avant de commencer
Les lignes	30 relations, 16 tables et 14 vues, 0 écart dans les deux sens
Les témoins	5 tests singuliers au vert ; 465 924 613,14 $ au fait et à l'ERP
L'incident	les semaines d'assemblage du 20 et du 27 octobre, 16 et 18 réclamations de refroidissement ; la vibration hors de sa bande du 21 au 28 octobre
La CI	au vert sur la branche : le linteur, puis `PASS=129` sur 130 nœuds
La livraison	`README.md` et `LIVRAISON.md` complets

© 2026 iAlexMG. Tous droits réservés.

Ce que le projet porte	Le chiffre
Les modèles	31, dont 3 `ephemeral` : 13 vues de `staging`, 14 tables de `marts`, 1 vue de `presentation`
Le reste du graphe	1 seed, 1 snapshot, 98 tests de données, 1 test unitaire, 14 sources, 1 `exposure`
Les faits	`fct_ventes` 80 618, `fct_enregistrements` 28 118, `fct_reclamations` 4 337, `fct_production` 29 782, `fct_disponibilite_actifs` 43 680, `fct_sorties` 201 542, `fct_codes_sortie` 1 155
Les dimensions	`dim_date` 1 551, `dim_concessionnaire` 205, `dim_produit` 68, `dim_vehicule` 29 782, `dim_client` 24 719, `dim_actif` 23, `dim_anomalie` 5

Ce que vous savez maintenant et que vous ne saviez pas au Cours 08 : un projet n'est fini que le jour où on l'a effacé et refait ; une reconstruction se protège par une garde, parce que l'histoire qu'elle rejoue s'abîme sans bruit ; refaire n'est pas prouver, et une vue ne prouve rien par ses lignes ; un témoin compare l'étoile à autre chose qu'elle-même ; et une étoile bien conformée retrouve un incident par n'importe lequel de ses bouts.

La formation s'arrête ici. Votre étoile est ce que Business Intelligence restitue, ce que Data Science et Machine Learning Engineering modélisent, et ce qu'un agent d'AI Engineering interroge ; le kit 03 la rebâtit pour qui arrive sans elle. Et le dépôt `02_ae`, étiqueté `v1.0`, est la pièce que vous pouvez montrer.

Dépannage

Le message ou le symptôme	Ce qu'il veut dire	La parade
`exiger_entrepot_neuf : l'entrepot porte deja le projet`	la garde a trouvé un objet du projet dans l'entrepôt	c'est voulu avant l'étape 5 ; après, un schéma a échappé à l'effacement : relancer le `drop schema` qui manque
`'reconstruire.cmd' n'est pas reconnu`	le terminal est PowerShell, qui ne lance pas un script du dossier courant sans chemin	`.\reconstruire.cmd`, ou l'Anaconda Prompt
`Catalog Error: Schema with name staging does not exist!` au `drop schema`	le schéma est déjà effacé	passer au suivant, puis vérifier la liste des schémas

© 2026 iAlexMG. Tous droits réservés.

Le message ou le symptôme	Ce qu'il veut dire	La parade
`fatal: destination path '02_ae_neuf' already exists and is not an empty directory.`	une copie d'un essai précédent est restée	`rmdir /s /q 02_ae_neuf`, puis recloner
La reconstruction rend ce que le dépôt de travail ne rend pas	un changement n'est pas commité, et la copie ne le voit pas	`git status` dans `02_ae`, commiter, recloner
`IO Error: Cannot open file ... utilisé par un autre processus`	la ligne de commande DuckDB est restée ouverte	`.exit` avant toute commande dbt ou tout script
`IO Error: Cannot open database "...boreal_avant.duckdb" in read-only mode: database does not exist` à la comparaison	la copie de l'étape 5 n'existe pas, ou le chemin est faux	lancer le script depuis `Projet_Boreal` ; sans copie, la première preuve est perdue, les deux autres restent
`presente d'un seul cote`	une relation n'existe que d'un côté : une table que rien ne fabrique plus, ou un modèle neuf pas encore construit	lire son nom : une table orpheline se jette, un modèle manquant se construit
Toutes les vues sortent `DIFFERENTE`	l'entrepôt refait ne s'appelle plus `boreal.duckdb`, et chaque vue nomme sa base	refaire dans le même fichier, au même nom
`fin_avant_debut` au-dessus de 0 dans le snapshot	les lots ont été rejoués dans le désordre, ou par-dessus une histoire existante	effacer `snapshots`, puis relancer `reconstruire.cmd` sur l'entrepôt neuf
`gh: command not found`, ou pas de dépôt sur GitHub	la ligne de commande de GitHub n'est pas installée, ou le dépôt n'a pas de serveur distant	la branche se fusionne sur le poste, étape 12 ; la CI se répète sur une copie

© 2026 iAlexMG. Tous droits réservés.